

MASTER'S THESIS SUMMARY

From Experiment to Insight

Benchmarking HDF5/HSDS, Zarr, TileDB, and ROOT/TTree for large-scale
synchrotron and neutron-scattering data on AWS

Author: Casper Ove Kristiansson

Host: Scatterin AB

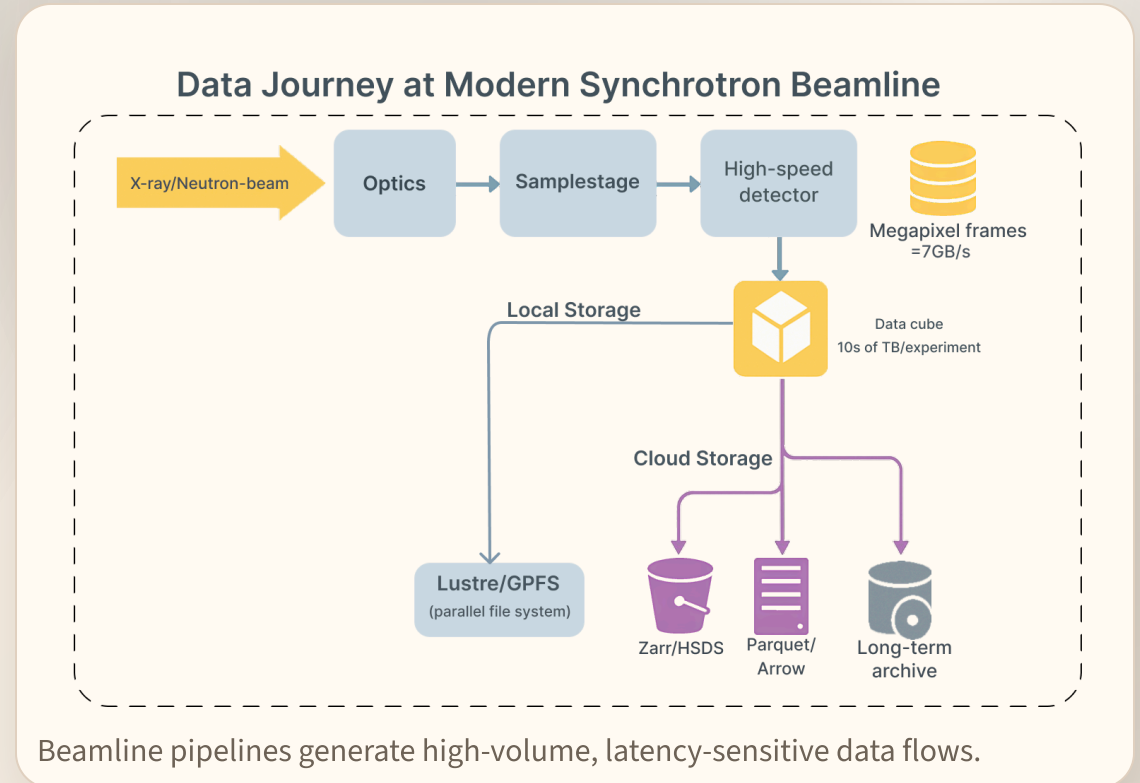
Affiliation: KTH EECS

Supervisors: Sebastian Dalleiger (KTH), Ahmet Bahadir Yildiz
(Scatterin AB)

Motivation: beamlines meet object storage

- Modern beamlines stream multi-GiB/s, turning single experiments into multi-TB datasets.
- Parallel file systems deliver throughput, but thousands of small reads suffer.
- S3 scales, yet per-request latency and request cost become visible.
- Need a side-by-side, cloud-native evaluation for slice latency vs full-scan throughput.

Goal: Identify storage + codec choices that reduce interactive latency while sustaining scan throughput.



Purpose & contributions

PROTOTYPE INGESTION

4 formats + 3 codecs

HDF5/HSDS, Zarr v3, TileDB, ROOT/TTTree (Parquet pilot only).

QUANTITATIVE BENCHMARKING

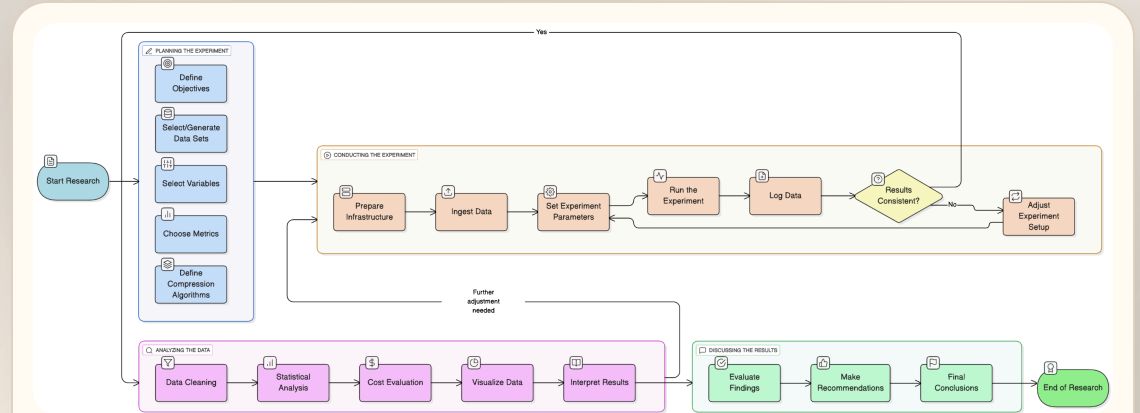
Slice + full scan

AWS S3 timings, request metrics, and compression footprints.

DECISION MATRIX

Latency-first vs throughput-first

Concrete, workload-driven recommendations.



Design-science cycle: plan → run → analyze → decide.

Research questions & scope

RQ1: For random single-frame reads on S3, which backend + codec minimize GM latency and control p95?

RQ2: For full-dataset scans, which backend + codec maximize GM throughput with stable p95?

RQ3: How do layout + codec choices change GETs/GiB and stored size, and explain timings?

SCOPE & DELIMITATIONS

- AWS only (S3 + EC2).
- One dense detector stack (~20 GiB, ~1300 frames).
- Codecs: gzip, lz4, zstd.
- Workloads: single-frame slices + full scans.
- Results are descriptive, session-conditional.

Dataset & workloads

DATASET

~20 GiB detector stack

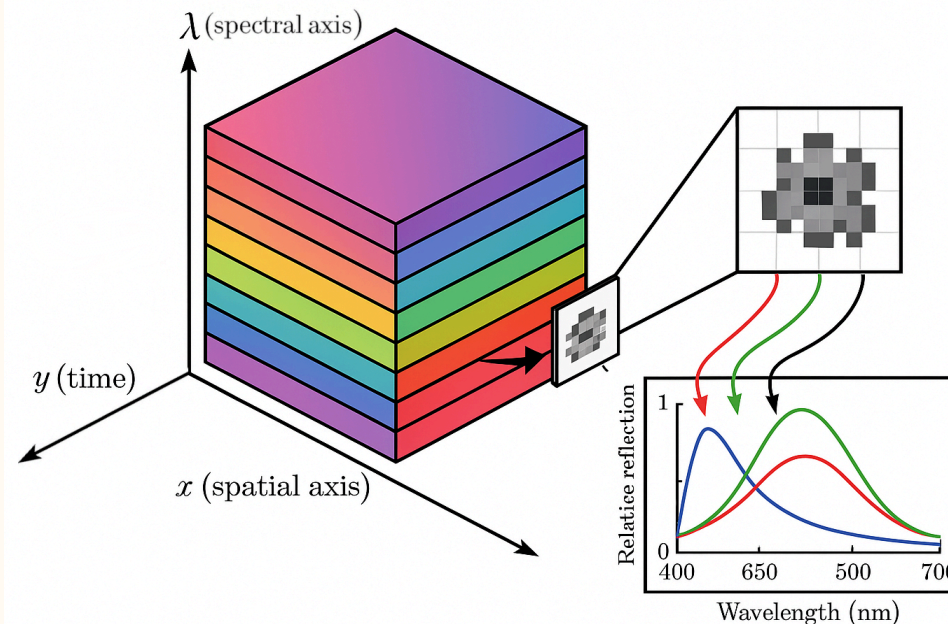
~1300 frames, 2880×2880, uint16 (~15.82 MiB/frame).

SOURCE

80 real CBF frames + synthetic augmentation (gain, contrast, jitter, noise).

WORKLOADS

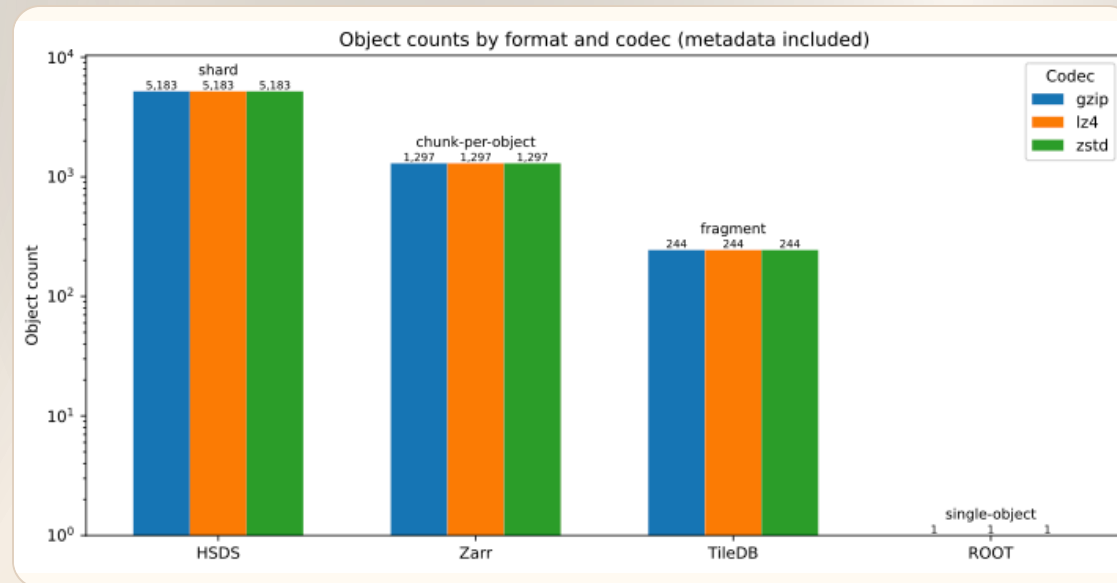
Random single-frame reads and full sequential scans.



Frame-aligned chunks preserve slice-level comparability.

Formats, codecs, layout parity

- HDF5 via HSDS: chunk-per-frame objects + metadata fan-out.
- Zarr v3: chunk-per-frame objects, direct S3 access.
- TileDB: frame-sized tiles, fragment-based layout.
- ROOT/TTTree: single object per codec (range-GET paging).
- Parquet pilot only: metadata open 3–4 min, single ~34 GiB file → excluded from timed runs.
- Codecs: gzip, lz4, zstd (same logical chunk size).



Objectization differs by layout (log scale).

RQ₁ — Interactive slice latency

FASTEST SLICE LATENCY

Zarr + zstd ≈ 155 ms

Zarr leads for zstd and lz4 slices.

GZIP EXCEPTION

TileDB ≈ 195 ms

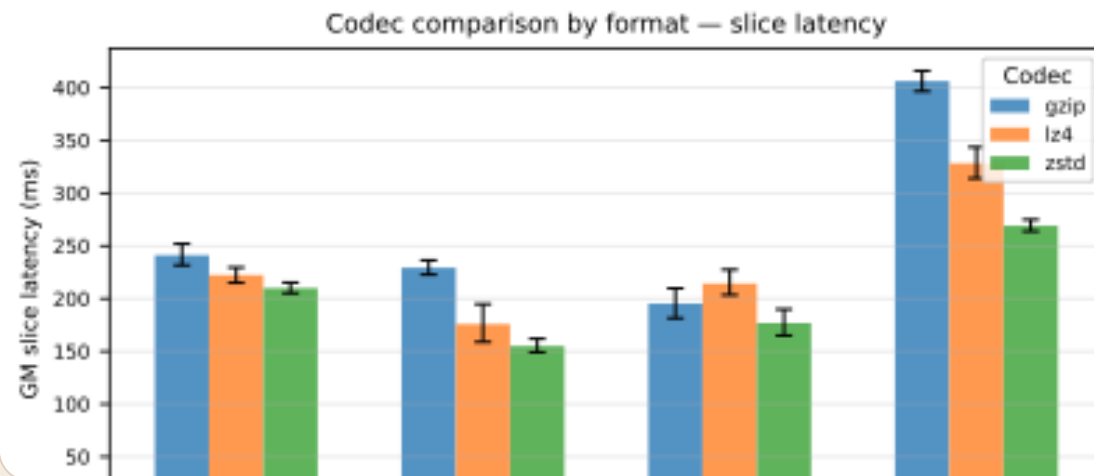
TileDB wins when gzip is mandated.

SLOWEST SLICES

ROOT ≈ 406 ms (gzip)

Single-object paging dominates despite few GETs.

Codec comparison by format (slice latency and full-scan throughput)



Top panel: GM slice latency across codecs.

RQ2 — Full-dataset throughput

THROUGHPUT LEADER

TileDB dominates

2.2–3.2× faster than Zarr across codecs.

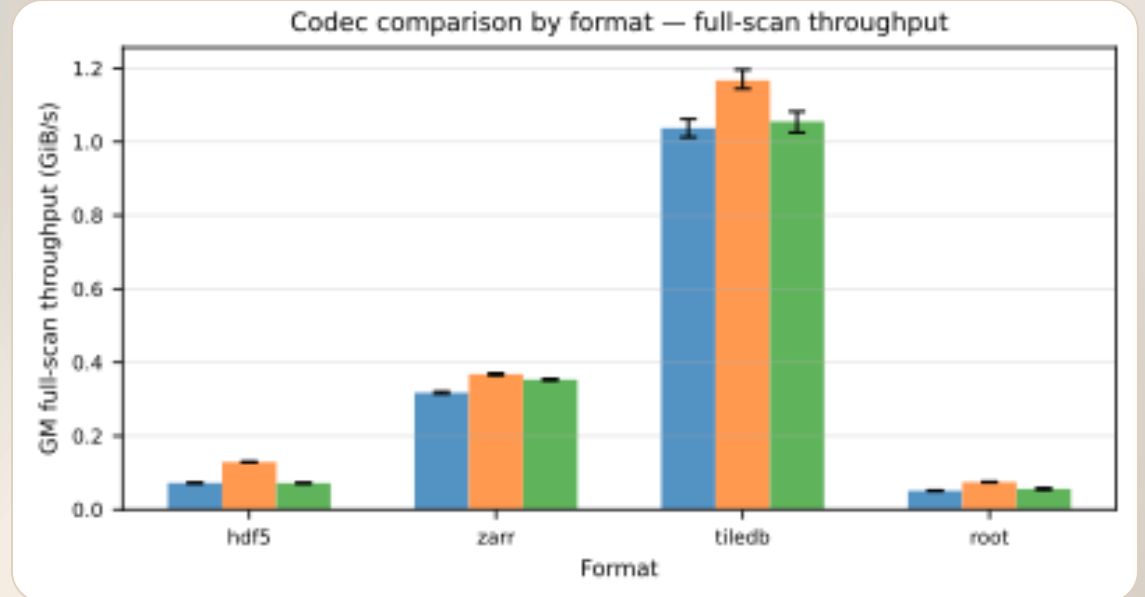
ZSTD FULL SCAN

10.4 s (1.06 GiB/s)

TileDB vs Zarr 30.4 s (0.35 GiB/s).

LAGGING FORMATS

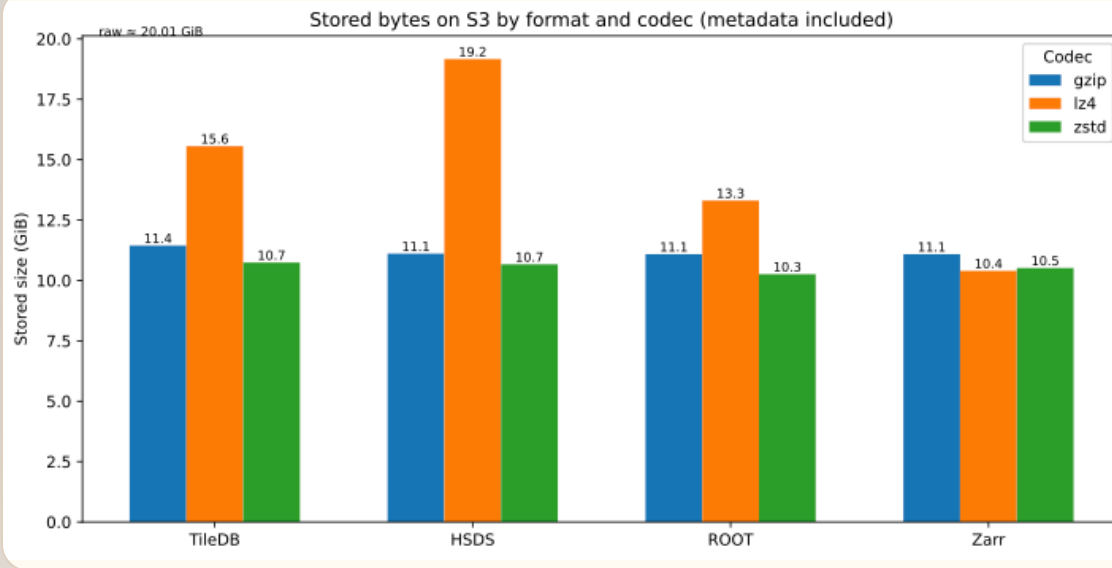
HSDS and ROOT remain far behind on scan throughput.



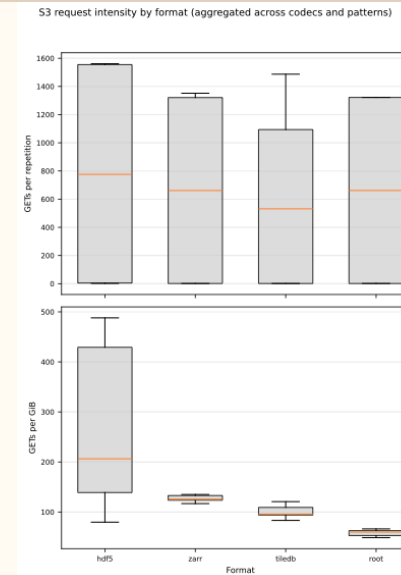
Bottom panel: GM full-scan throughput.

RQ3 — Cost posture & mechanisms

Key mechanisms: codec choice drives stored bytes, while layout drives GETs/GiB. ROOT keeps few GETs but still suffers high latency.



Stored size (codec-driven).



GETs per repetition and per GiB.

Decision matrix (practitioner choices)

LATENCY-FIRST (INTERACTIVE SLICES)

- Default: Zarr + zstd
- If gzip required: TileDB + gzip
- Avoid: ROOT for frame-random reads
- Caution: HSDS + lz4 (storage-heavy)

THROUGHPUT-FIRST (FULL SCANS)

- Default: TileDB + zstd
- Option: TileDB + lz4 if CPU-bound
- Second choice: Zarr for ecosystem fit

All rankings are descriptive and session-conditional for this dataset and testbed.

Status & remaining plan

CURRENT STATUS

- Main experimental work completed.
- Final revision and polishing phase (motivation, methods framing, limitations).
- Awaiting supervisor feedback.
- Supervisor: Sebastian Dalleiger.
- Target submission: late Feb 2026 (early March depending on feedback).

REMAINING PLAN

- Incorporate supervisor feedback.
- Active listening completed; opposition remaining.
- Prepare final defense presentation.

Repository: github.com/CasperKristiansson/KTH-MSc-Thesis-Computer-Science